CS 240 (Lab-9)
CNN Graded In-Lab

TAs: Abhi

Spring 2026

This in-lab is a continuation of the previous outlab on CNNs. In that outlab, you studied convolution, pooling, PyTorch CNNs, and architecture ideas such as BatchNorm and residual connections. In this lab test, you will apply those same ideas in a new but closely related setting: **1D CNNs for motif detection in time-series data**.

Story: Hamza and the Hidden Motif Signals

Hamza is helping build a small detector for noisy 1D communication traces recorded on campus. Each trace is of length 128. Somewhere inside the trace there may be a short local pattern, called a ***motif****. The location of the motif can change from trace to trace, and the signal also contains noise and drift. Hamza informally calls this effort* ***Project Dhurandhar****.*

The detector should identify useful local patterns even when their exact position changes from one signal to another.A model that depends too much on absolute position will not perform well. A CNN, however, can learn local filters that detect the same pattern at different positions.

Your task is to help Hamza build the detector pipeline step by step:

1. verify basic 1D filter behavior on toy traces,
2. prepare traces for the neural model,
3. build a baseline 1D CNN,
4. improve it using ideas from the outlab,
5. train and validate the improved detector.

So throughout this lab, all parts should be viewed as components of one single motif-detection pipeline.

Instructions

- Duration: **2 hours**
- This is a graded lab test.
- All Python files are in the same folder.
- The dataset is inside `data/`.
- Edit only the required source files.
- Do not rename files, functions, or classes.
- Do not change function signatures.
- Unless explicitly asked, do not print inside the required functions.
- Your code must run on CPU.

- The autograder will import your code. Therefore:
  - do not read files automatically at import time,
  - do not depend on hidden global state,
  - do not hardcode outputs.

Directory structure

```
lab_9/
|--- data
|   |--- train_labels.npy
|   |--- train_signals.npy
|   |--- val_labels.npy
|   |--- val_signals.npy
|--- local_runner.py
|--- motif_models.py
|--- motif_training.py
|--- public_checker.py
|--- signal_ops.py
|--- ts_cnn.py
```

Files You Should Edit

You are expected to edit only:

- `signal_ops.py`
- `motif_models.py`
- `motif_training.py`

Do not edit:

- `ts_cnn.py`
- `local_runner.py`
- `public_checker.py`
- anything inside `data/`

Which File Belongs to Which Part

Part	What Hamza is doing	File(s) to edit
Part A	verify toy filter behavior and prepare traces in the right shape	`signal_ops.py`
Part B	build the baseline motif detector	`motif_models.py`
Part C	improve the detector using BatchNorm and GAP	`motif_models.py`
Part D	train and validate the improved detector	`motif_training.py`
Bonus	try a residual version	`motif_models.py`

How to Test Locally

Inside the `lab9/` folder, you may run `python local_runner.py` and `python public_checker.py`

Difference between the two scripts:

- `local_runner.py` is more verbose and is meant for debugging.
- `public_checker.py` is shorter and stricter and acts like a public sanity check.

Submission Directory Structure

```
submission_{RollNumber}
|--- signal_ops.py
|--- motif_models.py
|--- motif_training.py
```

Use the `check-cs240` and `submit-cs240` scripts to submit.

Allowed Imports

Use only the ones already included, do not import any other external libraries:

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader
```

Optional Dataset Visualisation Utility

An **optional file** named `visualise.py` may be provided to help you inspect the training data visually. This file is **not part of the graded submission** and is meant only for local exploration. You can solve the problem and submit even without touching it. It can help you understand:

- how the raw 1D signals look,
- how signals differ across labels,

Problem Setting

Each signal is a 1D time-series of length 128.

There are 4 classes:

- Class 0: signal contains motif A
- Class 1: signal contains motif B
- Class 2: signal contains motif C
- Class 3: signal contains no target motif

The files inside `data/` are:

- `data/train_signals.npy`: shape `(N_train, 128)`, dtype `float32`
- `data/train_labels.npy`: shape `(N_train,)`, dtype `int64`
- `data/val_signals.npy`: shape `(N_val, 128)`, dtype `float32`
- `data/val_labels.npy`: shape `(N_val,)`, dtype `int64`

The label space is:

$$\{0, 1, 2, 3\}$$

Important Note on Convolution

Whenever you are asked to manually implement "1D convolution" in this lab test, use the **deep-learning convention**, i.e. **cross-correlation**.

That means:

- do not reverse the kernel,
- if `x_pad` is the padded signal and `w` is the kernel, then

$$y[t] = \sum_{i=0}^{K-1} x_{\text{pad}}[t \cdot S + i] \cdot w[i].$$

Project Dhurandhar begins with getting the low-level signal operations right.

1 Part A: Hamza Studies the Traces First (30 marks)

Edit file: `signal_ops.py`

To keep this test manageable in 2 hours, a few tiny formula helpers are already provided. In this part, you only need to implement the two core functions below.

A1. Manual 1D Convolution / Cross-Correlation

Implement: `manual_convolution_1d(x, kernel, stride=1, padding=0)`

Input:

- `x`: 1D NumPy array of shape `(L,)`
- `kernel`: 1D NumPy array of shape `(K,)`

Output:

- Return a 1D NumPy array of shape `(L_out,)`

Requirements:

- Use NumPy only.
- Use zero padding on both sides if `padding > 0`.
- Do not reverse the kernel.
- Do not mutate the inputs.

What not to do:

- Do not use `torch.nn.Conv1d`.
- Do not use SciPy.

A2. Prepare the Input Batch

Implement: `prepare_input_batch(x)`

This function must convert the input to a `torch.float32` tensor of shape `(B,1,L)`.

Accepted input shapes:

- `(L,)`
- `(B,L)`
- `(B,1,L)`

Accepted input types:

- NumPy array
- PyTorch tensor

Required behavior:

- `(L,)` becomes `(1,1,L)`
- `(B,L)` becomes `(B,1,L)`
- `(B,1,L)` remains unchanged
- dtype must become `torch.float32`

What not to do:

- Do not move data to GPU here.
- Do not normalize here.
- Do not accept arbitrary higher-dimensional inputs.

Now that the trace-processing utilities are ready, Hamza can build the first working detector.

2 Part B: Hamza Builds the First Detector (25 marks)

Edit file: `motif_models.py`

Implement: `class ShallowCNN1D(nn.Module)`

The architecture must be exactly:

Layer	Specification
Input	`(B,1,128)`
Conv1	`Conv1d(1,8,kernel_size=7,stride=1,padding=3)`
Act1	`ReLU`
Pool1	`MaxPool1d(2,2)`
Conv2	`Conv1d(8,16,kernel_size=5,stride=1,padding=2)`
Act2	`ReLU`
Pool2	`MaxPool1d(2,2)`
Flatten	flatten all non-batch dimensions
FC1	`Linear(16 * 32, 32)`
Act3	`ReLU`
FC2	`Linear(32,4)`

Implement both `__init__()` and `forward()`.

Requirements:

- forward must accept any valid input handled by `prepare_input_batch`,
- forward must return logits of shape `(B,4)`,
- do not apply softmax.

The baseline works, but Project Dhurandhar needs a detector that is more stable and less fragile.

3 Part C: Hamza Improves the Detector (15 marks)

Edit file: `motif_models.py`

Implement: `class BetterCNN1D(nn.Module)`

The improved architecture must be exactly:

Layer	Specification
Input	`(B,1,128)`
Conv1	`Conv1d(1,16,kernel_size=5,stride=1,padding=2)`
BN1	`BatchNorm1d(16)`
Act1	`ReLU`
Pool1	`MaxPool1d(2,2)`
Conv2	`Conv1d(16,32,kernel_size=5,stride=1,padding=2)`
BN2	`BatchNorm1d(32)`
Act2	`ReLU`
GAP	`AdaptiveAvgPool1d(1)`
Flatten	squeeze last dimension only
FC	`Linear(32,4)`

Implement both `__init__()` and `forward()`.

Why this part exists:

- BatchNorm reflects the architecture-improvement discussion from the outlab.
- Global average pooling reduces parameter count and makes the detector less dependent on exact position.

Requirements:

- forward must accept any valid input handled by `prepare_input_batch`,
- forward must return logits of shape `(B,4)`,
- do not apply softmax.

Once the improved detector is implemented, the final step is to train and validate it on archived traces.

4 Part D: Hamza Trains the Improved Detector (30 marks)

Edit file: `motif_training.py`

To keep the lab manageable, the small bookkeeping helpers are already provided:

- `count_parameters(model)`
- `_accuracy_from_logits(logits, y)`

You only need to complete the marked TODO sections inside:

```
train_better_model(train_x, train_y, val_x, val_y, ...)
```

This function must:

- construct a `BetterCNN1D`,
- use `nn.CrossEntropyLoss()`,
- use Adam with learning rate `lr`,
- train on the training set,
- evaluate validation accuracy after each epoch,
- return: `(model, history)` where `history` has keys: `"train_loss", "train_acc", "val_acc"`

Important note: Although the public data files are in `data/`, the autograder will pass arrays/tensors directly to your function. Therefore:

- do not read files inside `train_better_model`,
- use the function arguments directly.

What not to do:

- do not apply softmax before `CrossEntropyLoss`,
- do not train on the validation set,
- do not print inside the function.

5 Bonus (up to 10 marks)

Edit file: `motif_models.py`

Implement:

- `ResidualBlock1D`
- `ResidualCNN1D`

This is intentionally optional and is only for students who finish the main lab comfortably.